# Part I: Step-by-Step Tutorial: Writing Basic Unit Tests with JUnit (Java) and Pytest (Python)

## Introduction to JUnit (Java)

### Step 1: Set Up Your Development Environment

1. **Install Java Development Kit (JDK)**: Ensure that JDK 8 or higher is installed.
2. **Install an Integrated Development Environment (IDE)**: IntelliJ IDEA or Eclipse.
3. **Add JUnit Dependency**:
    - If using Maven, add the following dependency to your `pom.xml` file:
    - `<dependency>`
    - `    <groupId>org.junit.jupiter</groupId>`
    - `    <artifactId>junit-jupiter-api</artifactId>`
    - `    <version>5.8.1</version>`
    - `    <scope>test</scope>`
      `</dependency>`

### Step 2: Create a Simple Java Class to Test

Create a class `Calculator` with basic methods to test:

```
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }

    public int subtract(int a, int b) {
        return a - b;
    }
}
```

### Step 3: Write Unit Tests with JUnit

Create a test class `CalculatorTest`:

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;

public class CalculatorTest {
    @Test
    public void testAdd() {
        Calculator calculator = new Calculator();
        assertEquals(5, calculator.add(2, 3));
        assertEquals(0, calculator.add(2, -2));
    }
```

```
    @Test
    public void testSubtract() {
        Calculator calculator = new Calculator();
        assertEquals(1, calculator.subtract(3, 2));
        assertEquals(-5, calculator.subtract(0, 5));
    }
}
```

### Step 4: Run Your Tests

- **In IntelliJ IDEA**: Right-click the `CalculatorTest` class and select **Run 'CalculatorTest'**.
- **In Eclipse**: Right-click the file and choose **Run As > JUnit Test**.

### Step 5: Analyze the Results

- Ensure all tests pass.
- Adjust code or tests to cover edge cases such as negative numbers or zero.

---

# Introduction to Pytest (Python)

### Step 1: Set Up Your Development Environment

1. **Install Python**: Ensure Python 3.x is installed.
2. **Install Pytest**:

   ```
   pip install pytest
   ```

3. **Create a Python Project**: Use any code editor, such as PyCharm or VSCode.

### Step 2: Create a Simple Python Module to Test

Create a file `calculator.py`:

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b
```

### Step 3: Write Unit Tests with Pytest

Create a test file `test_calculator.py`:

```
import pytest
```

```
from calculator import add, subtract

def test_add():
    assert add(2, 3) == 5
    assert add(2, -2) == 0

@pytest.mark.parametrize("a, b, expected", [
    (3, 2, 1),
    (0, 5, -5),
    (-3, -2, -1)
])
def test_subtract(a, b, expected):
    assert subtract(a, b) == expected
```

### Step 4: Run Your Tests

- **Run all tests**:

  ```
  pytest
  ```

- **Run specific tests**:

  ```
  pytest test_calculator.py::test_add
  ```

### Step 5: Analyze the Results

- Pytest will display results in the terminal.
- Modify the `calculator.py` or `test_calculator.py` as needed to cover additional edge cases.

# Part II: Exercises

**Exercise 1: Basic Arithmetic Operations**

• Create a class that performs multiplication and division.

**• Instructions:**

- Ensure that your class methods handle edge cases such as dividing by zero and multiplication with large numbers.
- Include comments explaining the logic within the code. • Write unit tests to check for:
- Normal cases (e.g., 6 * 7, 42 / 7).
- Zero cases (e.g., 0 * 5, 10 / 0).
- Negative number cases (e.g., -3 * 4, -15 / 5).

**Exercise 2: String Manipulation**

• Create a function that reverses a string and returns it.

• Instructions:

- Ensure your function handles special characters and whitespace correctly.
- Add comments to describe the approach taken. • Write tests for:
- Normal strings (e.g., "hello").
- Empty strings (e.g., "").
- Strings with special characters (e.g., "!@#", "race car").

**Exercise 3: Array/List Operations**

• Create a method that finds the maximum number in an array. • Instructions:

- Ensure your method handles arrays with various sizes, including large arrays.
- Explain any algorithms used in comments. • Write tests for:
- Arrays with positive numbers (e.g., [1, 3, 5, 7]).
- Arrays with negative numbers (e.g., [-1, -3, -5, -7]).
- Mixed number arrays (e.g., [-3, 0, 9, -2]).
- Edge cases such as arrays with one element (e.g., [5]) or no elements ([]).

**Exercise 4: User Input Validation**

• Write a function that validates email addresses.

• Instructions:

- Ensure your function checks for standard email format (e.g., name@domain.com).
- Include comments explaining validation rules. • Create unit tests to handle:
- Valid email addresses (e.g., "test@example.com").
- Invalid email formats (e.g., "test@@example", "test.com").
- Edge cases (e.g., empty strings, "@").

**Exercise 5: Exception Handling**

• Implement a function that divides two numbers and raises an exception if the divisor is zero.

• Instructions:

- Ensure your function includes clear exception messages.
- Add comments to clarify exception handling logic. • Write unit tests to check:
- Normal division (e.g., 10 / 2).
- Division by zero to ensure exceptions are raised correctly.
- Edge cases such as division involving negative numbers (e.g., -10 / 5).

**Exercise 6: Data Transformation**

• Create a method that converts a list of dictionaries into a CSV string.

• Instructions:

- Ensure your method handles various data types and edge cases.
- Comment on how the method formats the CSV string. • Write tests to validate:
- Correct CSV formatting (e.g., [{"name": "John", "age": 30}]).
- Handling empty lists (e.g., []).
- Handling unexpected data types (e.g., missing keys, non-dictionary items)